Recover the Design

Author: Author

Reviewer: Reviewer (review pending)

Group number: withheld

Date: 2026-10-21

Abstract

This project restores an axis-aligned fence layout from connector degrees. We build candidate segments between consecutive connectors, forbid crossings at empty positions, and solve the resulting binary constraints by propagation and exhaustive backtracking. The C++17 implementation passes 935 local cases and the same suite under AddressSanitizer and UndefinedBehaviorSanitizer. The method is exact but has exponential worst-case time. This is a first draft; independent review and hidden-judge acceptance remain pending.

Contents

1 Introduction **3**

2 Data Structure / Algorithm Specification **3**

2.1 Candidate Segments and Constraints 3

2.2 Propagation and Search 3

2.3 Correctness 4

3 Testing Results **5**

3.1 Environment and Methodology 5

3.2 Correctness Results 5

3.3 Timing Results 6

4 Analysis and Comments **6**

4.1 Time Complexity 6

4.2 Space Complexity 7

4.3 Limitations and Improvements 7

5 Conclusion **7**

6 References **8**

7 Author List **8**

8 Declaration **8**

9 Signatures **8**

10 Source Code Appendix **8**

1 Introduction

The input is an $n \times m$ grid, where $1 < n, m \leq 50$. A positive entry specifies a connector degree from 1 to 4; zero means no connector. Fences must be horizontal or vertical. Turns and intersections require connectors [1]. The output gives each connector's row, column, and four direction flags, in row-major order. If no design exists, the program prints `No Solution`.

There is no global connectivity requirement in the supplied statement. Separate components and cycles are allowed. A fence may continue straight through a connector by using two opposite directions. An all-zero grid has an empty valid output. The supplied screenshots do not show time or memory limits, and the sample output is truncated; we validate solutions geometrically instead of comparing output strings.

2 Data Structure / Algorithm Specification

2.1 Candidate Segments and Constraints

Create one vertex for each nonzero grid position. In each row and column, connect consecutive vertices by a candidate edge. A more distant connector cannot be reached in one segment without passing another connector; such a fence is represented by multiple consecutive edges. Each vertex has at most four candidates. Each edge stores its endpoints and their corresponding direction indices.

For each candidate edge e, let $x_e \in \{0, 1\}$ indicate whether it is selected. At vertex v, the requirement is $\sum_{e \ni v} x_e = k_v$. For horizontal and vertical candidates whose interiors intersect at a zero cell, require $x_e + x_f \leq 1$. These constraints encode the full task: endpoints are connectors, segments are straight, degrees are exact, and empty-cell crossings are forbidden.

Row and column scans build the edges. Two temporary grids record the horizontal and vertical edge crossing each empty cell. If both are present, record a conflict. Each empty cell belongs to at most one horizontal and one vertical candidate interior.

2.2 Propagation and Search

The state of each edge is unknown, excluded or selected. For each vertex, count selected edges s and unknown edges u. Reject the state if $s > k_v$ or $s + u < k_v$. If $s = k_v$, exclude all unknown incident edges. If $s + u = k_v$, select all unknown incident edges. Selected edges exclude every conflicting edge. Repeat full scans until no state changes.

If unknown edges remain, choose a vertex with the fewest unknown incident edges and branch on one of those edges. Try selection, then exclusion. A trail stores every newly assigned edge. After

a failed recursive call, restore all entries after the saved trail boundary. This avoids copying the entire state at each branch.

```
search:
    propagate degree and crossing constraints
    if contradiction: return failure
    if every edge is assigned: return success
    choose an unknown edge at a most-constrained vertex
    save the trail boundary
    for value in [selected, excluded]:
        assign the edge
        if search succeeds: return success
        roll back to the saved boundary
    return failure
```

2.3 Correctness

Candidate completeness: split every valid fence at each encountered connector. Each resulting segment joins consecutive connectors in one row or column, and therefore appears in the candidate list. Conversely, each candidate is a straight segment with no connector in its interior.

Propagation safety: the two failure inequalities mean that a degree cannot be met. If the required degree is already reached, every remaining edge must be excluded. If all unknown edges are needed to reach that degree, every one must be selected. A selected edge's crossing conflicts must be excluded. Thus each propagation step preserves every possible completion of the current branch.

Search completeness: a chosen unknown edge has exactly two possible values, and the search explores both. Rollback restores precisely the parent assignment, including changes made during propagation. Each recursive step fixes at least one new edge, so every branch terminates. By induction on the number of unknown edges, failure means neither value has a feasible completion.

Output soundness: a successful leaf has no unknown edges and has passed all degree and crossing checks. Its selected segments therefore satisfy every stated rule. Endpoint direction flags are generated from the same selected edge, so they agree at both ends. Vertices are stored in input order, giving the required output order. Together these arguments show that the program prints a valid design exactly when one exists, subject to sufficient execution resources.

3 Testing Results

3.1 Environment and Methodology

Local runs were performed on 2026-10-21 using Linux 6.6.87.2-microsoft-standard-WSL2, an AMD Ryzen 7 H 255 processor, GCC 13.3.0 and Python 3.13.13. The optimized executable uses C++17 and `-O2 -Wall -Wextra -Wpedantic`. The sanitizer build uses `-O1 -g`, AddressSanitizer and UndefinedBehaviorSanitizer, with frame pointers and non-PIE linking.

The Python checker independently traces each printed direction to the first connector. It checks coordinates, order, spacing, degree totals, reciprocal flags, boundaries and empty-cell crossings. For small inputs, a separate oracle constructs candidate pairs geometrically and enumerates every edge subset. This oracle checks both false positive solutions and incorrect `No Solution` results. Three deliberately invalid outputs also test the validator's rejection behavior.

3.2 Correctness Results

Case family	Cases	Observed result
Supplied sample input	1	Valid recovered design
Every 2 by 2 grid with entries 0 to 4	625	Agreement with exhaustive oracle
Seeded random 3 by 3 grids	200	Agreement with exhaustive oracle
Crossing, junction, multiple-solution and straight-through cases	4	All expected outcomes
Planted valid 4 by 4 designs	100	All recovered designs valid
Structured 50 by 50 boundary cases	5	All expected outcomes

The optimized build and the sanitizer build each passed all 935 solver cases and all three validator rejection checks. No sanitizer diagnostic was observed. Random generation uses seed 17. These checks are local evidence, not independent reviewer approval or PTA acceptance.

3.3 Timing Results

Each structured 50 by 50 input has one warmup run followed by ten measured runs. The table reports milliseconds for the entire Python wrapper interval: process startup, input transfer, solving, output transfer, and Python validation. These are not isolated algorithm CPU times. Benchmarking was repeated after the sanitizer run finished; the table uses that later run.

Input	Median (ms)	Minimum to maximum (ms)
Empty grid	1.868	1.392 to 2.131
Degree 1 at every position	41.276	38.732 to 42.976
Full grid with all neighboring segments required	8.283	7.785 to 10.294
Degree 1 at both ends of each row	2.292	1.932 to 2.749
Impossible degree 4 at every position	2.520	2.085 to 2.909

All five inputs have known expected feasibility. The degree-1 grid admits many matchings and exercises branching. The full-neighbor grid forces every edge; the degree-4 grid fails at a corner. These structured cases do not measure the most difficult ambiguous or infeasible search instances. Local timing records are generated by the reproduction command and are not included in the submission bundle.

4 Analysis and Comments

4.1 Time Complexity

Let V be the connector count, E the candidate count and X the number of crossing pairs. Since each connector has at most four candidates, $E \leq 2V$. Since each empty cell contributes at most one crossing pair, $X \leq nm$. Building the model takes $O(nm + V + E + X)$ time: the candidate interiors within each row or column are disjoint, so their total marking work is linear in the grid area.

One propagation scan takes $O(V + E + X)$. Every nonfinal successful scan fixes at least one previously unknown edge, so a propagation call uses at most $E + 1$ scans. Each branch fixes an unknown edge, giving at most $2^{E+1} - 1$ nodes in a full binary search tree. A conservative worst-case bound is therefore $O(nm + 2^E(E + 1)(V + E + X))$. These bounds follow directly from this

implementation and the preceding counting arguments; they do not imply polynomial running time on all allowed inputs.

4.2 Space Complexity

The grid, graph and conflict lists use $O(nm+V+E+X)$ total space. The assignment trail holds at most E entries along the active search path, and recursive search depth is at most $E+1$. Updates and rollback therefore need $O(E)$ auxiliary space. No full graph copy is made per recursive level.

4.3 Limitations and Improvements

1. Backtracking has exponential worst-case time. Add harder adversarial cases and obtain actual PTA time and memory limits before treating this draft as judge-ready.
2. Propagation rescans the full model. A queue of affected vertices and conflicts could avoid repeated work; this optimization has not been implemented or benchmarked.
3. The current performance study covers five structured maximum-size cases. Broader sparse, dense and near-infeasible cases are needed to characterize practical search cost.

5 Conclusion

The implementation models the geometric restrictions explicitly and recovers valid layouts on the supplied sample and the local test suite. An independent exhaustive oracle supports small-instance correctness, and sanitizer checks found no memory or undefined-behavior errors in the tested cases. Independent review and hidden-judge evaluation remain outstanding.

6 References

[1] Y. Chen, "8-1 Recover the Design," Zhejiang University, PTA assignment statement, screenshots supplied by the user, Oct. 21, 2026. Local transcription: `problem.md`. The screenshots are the source for the task requirements; the correctness and complexity arguments above are derived for this implementation.

7 Author List

Author is responsible for code, tests, report preparation and responses to review comments.

Reviewer is responsible for independent reproduction and approval. This role assignment does not indicate that review has occurred.

8 Declaration

This draft records the current implementation and local checks for "Recover the Design." The final course declaration must be confirmed by the group before final submission.

9 Signatures

Author: pending confirmation.

Reviewer: review and signature pending.

10 Source Code Appendix

The complete solver is provided in the accompanying `code/main.cpp`. The independent checker is in `scripts/check.py`, and the supplied sample input is in `tests/sample1.in`. Build and reproduction commands are listed in the accompanying README.